

CLIENT

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080
#define MAX 20
int main()
{
    int sock = 0;
    struct sockaddr_in serv_addr;
    int n, matrix[MAX][MAX];
    char result[50];
    sock = socket(AF_INET, SOCK_STREAM, 0);
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr);
    connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr));
    printf("Enter order of matrix: ");
    scanf("%d", &n);
    srand(time(0));
    printf("\nEnter matrix elements:\n");
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            scanf("%d", &matrix[i][j]);
        }
    }
    printf("\nEnter matrix:\n");
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            printf("%4d", matrix[i][j]);
        }
        printf("\n");
    }
    write(sock, &n, sizeof(int));
    write(sock, matrix, sizeof(matrix));
    read(sock, result, sizeof(result));
    printf("\nMatrix Type Received from Server: %s\n", result);
    close(sock);
    return 0;
}
```

SERVER

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080
#define MAX 20
int main()
{
    int server_fd, new_socket;
    int i,j;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    int n, matrix[MAX][MAX];
    char result[50];
    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);
    bind(server_fd, (struct sockaddr *)&address, sizeof(address));
    listen(server_fd, 3);
    printf("Waiting for client connection...\n");
    new_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t *)&addrlen);
    read(new_socket, &n, sizeof(int));
    read(new_socket, matrix, sizeof(matrix));
    int upper = 1, lower = 1, diagonal = 1;
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            if(i > j && matrix[i][j] != 0)
                upper = 0;
            if(i < j && matrix[i][j] != 0)
                lower = 0;
            if(i != j && matrix[i][j] != 0)
                diagonal = 0;
        }
    }
    if(diagonal)
    {
        strcpy(result, "Diagonal Matrix");
        printf("\n diagonal matrix:\n");
        for(i=0;i<n;i++)
        {
            for(j=0;j<n;j++)
```

```

    {
        if(i==j)
        {
            printf("%4d",matrix[i][j]);
        }
        else
        {
            printf("%4d",0);
        }
    }
    printf("\n");
}
}
else if(upper)
{
    strcpy(result, "Upper Triangular Matrix");
    printf("\nupper triangular matrix:\n");
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            if(i<=j)
            {
                printf("%4d",matrix[i][j]);
            }
            else
            {
                printf("%4d",0);
            }
        }
        printf("\n");
    }
}
else if(lower)
{
    strcpy(result, "Lower Triangular Matrix");
    printf("\nlower triangular matrix:\n");
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            if(i>=j)
            {
                printf("%4d",matrix[i][j]);
            }
        }
    }
}

```

```

        else
        {
            printf("%4d",0);
        }
    }
    printf("\n");
}
}
else
    strcpy(result, "Neither Upper nor Lower nor Diagonal");
write(new_socket, result, sizeof(result));
printf("Matrix Type Sent to Client: %s\n", result);
close(new_socket);
close(server_fd);
return 0;
}

```

OUTPUT

CASE 1

Client

Enter order of matrix: 3

Enter matrix elements:

1 0 0 5 0 0 9

Entered Matrix:

1 0 0

0 5 0

0 0 9

Matrix Type Received from Server: Diagonal Matrix

Server

Waiting for client connection...

Diagonal Matrix:

1 0 0

0 5 0

0 0 9

Matrix Type Sent to Client: Diagonal Matrix

CASE 2

Client

Enter order of matrix: 3

Enter matrix elements:

1 2 3 0 4 5 0 0 6

Entered Matrix:

1 2 3

0 4 5

0 0 6

Matrix Type Received from Server: Upper Triangular Matrix

Server

Waiting for client connection...

Upper Triangular Matrix:

1 2 3
0 4 5
0 0 6

Matrix Type Sent to Client: Upper Triangular Matrix

CASE 3

Client

Enter order of matrix: 3

Enter matrix elements:

1 0 0 2 3 0 4 5 6

Entered Matrix:

1 0 0
2 3 0
4 5 6

Matrix Type Received from Server: Lower Triangular Matrix

Server

Waiting for client connection...

Lower Triangular Matrix:

1 0 0
2 3 0
4 5 6

Matrix Type Sent to Client: Lower Triangular Matrix

CASE 4

Client

Enter order of matrix: 3

Enter matrix elements:

1 2 4 5 6 7 8 9

Entered Matrix:

1 2 3
4 5 6
7 8 9

Matrix Type Received from Server: Neither Upper nor Lower nor Diagonal

Server

Waiting for client connection...

Matrix Type Sent to Client: Neither Upper nor Lower nor Diagonal